

JavaScript & React Interview Questions

Explanations with examples for common JavaScript and React interview questions.

JavaScript Questions

Explain var, let, and const.

Answer: var is function-scoped, let and const are block-scoped. const cannot be reassigned.

Explanation: let and const were introduced in ES6 to provide better scoping rules and prevent accidental variable redeclaration.

Example:

```
var x = 1
let y = 2
const z = 3
```

What is Hoisting?

Answer: Hoisting is JavaScript's behavior of moving variable and function declarations to the top of their scope before execution.

Explanation: Only declarations are hoisted, not initializations.

Example:

```
console.log(a)
var a = 5
```

What is the Temporal Dead Zone (TDZ)?

Answer: The time between entering scope and variable declaration where let and const cannot be accessed.

Explanation: It prevents accessing variables before initialization.

Example:

```
console.log(a)
let a = 5
```

What is a Closure?

Answer: A closure is when a function remembers variables from its outer scope even after the outer function has finished execution.

Explanation: Closures allow data privacy and are used in callbacks and functional programming.

Example:

```
function outer(){
  let count = 0
  return function(){
    count++
  }
}
```

```
    console.log(count)
  }
}
```

What is Currying?

Answer: Currying transforms a function with multiple arguments into a sequence of functions each taking one argument.

Explanation: It helps create reusable functions and functional programming patterns.

Example:

```
function multiply(a){
  return function(b){
    return a*b
  }
}
```

JavaScript is single-threaded, so how does it handle asynchronous tasks?

Answer: It uses the event loop and Web APIs to offload async operations like timers or network requests.

Explanation: The main thread stays free while async tasks run in the background.

What is the Event Loop?

Answer: The event loop continuously checks the call stack and task queue to execute asynchronous callbacks.

Explanation: It ensures non-blocking behavior in JavaScript.

What are Microtasks and Macrotasks?

Answer: Microtasks include Promise callbacks, while macrotasks include setTimeout or setInterval.

Explanation: Microtasks are executed before macrotasks after the call stack is empty.

How do Promises work?

Answer: A Promise represents a future value that may be fulfilled or rejected.

Explanation: It allows asynchronous operations without deeply nested callbacks.

Example:

```
fetch('/api')
  .then(res => res.json())
  .then(data => console.log(data))
```

Difference between fetch and axios?

Answer: Fetch is a built-in browser API while Axios is a third party library.

Explanation: Axios automatically transforms JSON and supports interceptors.

What is Callback Hell?

Answer: Callback hell occurs when multiple nested callbacks make code difficult to read and maintain.

Explanation: Promises and async/await help avoid it.

What is Event Bubbling?

Answer: Event bubbling means events propagate from child elements up to parent elements.

Explanation: It allows handling events at higher levels of the DOM.

Difference between == and ===

Answer: == compares values with type conversion, while === compares values and types strictly.

Difference between null and undefined

Answer: undefined means a variable is declared but not assigned. null represents intentional absence of value.

What is Optional Chaining?

Answer: Optional chaining allows safe access to nested object properties without throwing errors.

Explanation: user?.address?.city

What is Destructuring?

Answer: Destructuring extracts values from arrays or objects into variables.

Explanation: const {name, age} = user

What is the Spread Operator?

Answer: It expands elements of arrays or objects.

Explanation: const arr2 = [...arr1]

React Questions

What is React Virtual DOM?

Answer: A lightweight copy of the real DOM used by React to detect changes efficiently.

Explanation: It improves performance by minimizing direct DOM manipulation.

What is Reconciliation?

Answer: The process React uses to compare the previous Virtual DOM with the new one and update the real DOM efficiently.

What is a Single Page Application (SPA)?

Answer: An SPA loads a single HTML page and dynamically updates content without reloading the page.

Difference between Props and State?

Answer: Props are read-only inputs passed to components. State is managed inside the component and can change.

What is Lifting State Up?

Answer: Moving shared state to the closest common parent so multiple components can access it.

What is the key prop in React?

Answer: Keys help React identify elements in lists and optimize rendering.

What are React Hooks?

Answer: Hooks allow functional components to use state and lifecycle features.

Explanation: Common hooks include `useState`, `useEffect`, `useContext`, `useMemo`, and `useCallback`.

How can we optimize performance in React?

Answer: Use memoization, lazy loading, code splitting, and avoid unnecessary re-renders.

What is React.memo?

Answer: A higher-order component that prevents unnecessary re-renders by shallowly comparing props.

Difference between `useMemo` and `useCallback`?

Answer: useMemo memoizes values, while useCallback memoizes functions.

What is the dependency array in useEffect?

Answer: It controls when the effect runs based on changes in dependencies.

What happens if useEffect has no dependency array?

Answer: The effect runs after every render.

What are Controlled vs Uncontrolled Components?

Answer: Controlled components manage form state in React, while uncontrolled components store state in the DOM.

What is Redux?

Answer: A predictable state management library used for managing global application state.

Redux vs Context API?

Answer: Redux provides centralized state management with middleware support, while Context is simpler and built into React.

Difference between CSR and SSR

Answer: CSR renders content in the browser, while SSR renders content on the server before sending HTML.

What is Hydration in React?

Answer: Hydration is the process of attaching React event listeners to server-rendered HTML.